

MINI HOSPITAL / PATIENT RECORD SYSTEM

Student Name	Aashita Singh
Registration No.	26BOE10030
Program	B.tech Bioengineering

Domain

Hospital / Patient Record Management

Table of Contents

1	Project Overview
2	Problem Statement
3	Objectives
4	Scope of the System
5	Key Features
6	Technology Stack
7	System Architecture
8	Data Model and Class Design
9	Detailed Class Documentation
10	Functional Module Documentation
11	Program Workflow
12	Menu and User Interaction
13	Input Validation and Error Handling
14	Sample Execution
15	Algorithm / Pseudocode
16	Complexity Analysis
17	Testing and Test Cases
18	Security and Data Considerations
19	Limitations
20	Future Enhancements
21	Conclusion
22	Complete Source Code Appendix

1. Project Overview

The Mini Hospital / Patient Record System is a console-based Python application designed to manage basic patient information and consultation records. The application demonstrates how object-oriented programming can be applied to a small healthcare record-management workflow.

The system maintains patient records in an in-memory dictionary. Each patient is represented as an object of the **Patient** class, while the **HospitalRecordSystem** class manages operations such as adding, searching, displaying, updating, and deleting patient records.

The application is intentionally lightweight. It does not use a database, web server, graphical interface, authentication system, or external healthcare API. Its primary purpose is to demonstrate core Python programming and object-oriented design concepts in a realistic domain.

High-Level Workflow

The user starts the application and is presented with a numbered menu. The selected operation is executed by the **HospitalRecordSystem** object. Patient data is stored in memory during the current execution of the program.

Application flow: **Start** → **Create HospitalRecordSystem** → **Display Menu** → **Read User Choice** → **Execute Operation** → **Return to Menu** → **Exit**.

2. Problem Statement

Hospitals and clinics need to maintain patient information in an organized manner. Even a small outpatient workflow requires information such as patient identity, age, gender, assigned doctor, department, and consultation status.

For a learning-oriented system, manually handling these details without a structured program can lead to inconsistent records and makes basic operations such as searching, updating, and deleting difficult.

This project addresses the problem by providing a simple menu-driven application that centralizes patient records and exposes common record-management operations through reusable methods.

3. Objectives

- Create a structured digital record for every patient.
- Assign a unique patient ID to each record.
- Store patient demographic and consultation information.
- Search patient records using Patient ID or exact patient name.
- Display individual or all patient records in a readable format.
- Update the assigned doctor and department.
- Update consultation status using predefined or custom values.
- Delete an existing patient record.
- Demonstrate object-oriented programming through classes and methods.
- Provide a simple foundation that can later be extended into a database or web application.

4. Scope of the System

In Scope	Out of Scope
Patient record creation	Real hospital deployment
Patient search	Medical diagnosis

In Scope	Out of Scope
Doctor/department assignment	Prescription management
Consultation status tracking	Billing and insurance
Record display	Laboratory information systems
Record deletion	User authentication/authorization
Console-based interaction	Persistent database storage

The project is therefore best understood as an educational prototype rather than a production healthcare information system.

5. Key Features

Feature	Description
Add Patient	Creates a Patient object and stores it using a unique patient ID.
Duplicate-ID Prevention	Rejects a new patient when the entered ID already exists.
Search Patient	Searches by exact patient ID or exact name, ignoring letter case.
Display All	Displays every patient currently stored in memory.
Doctor Assignment	Changes the assigned doctor and/or department.
Consultation Status	Supports Scheduled, In Progress, Completed, Cancelled, or custom status text.
Delete Record	Removes a patient from the dictionary using the patient ID.
Menu Loop	Keeps the application running until the user selects Exit.

6. Technology Stack

Technology / Concept	Use in Project
Python 3	Application programming language
OOP	Patient and HospitalRecordSystem classes
Dictionary	In-memory patient record storage
Input / Output	Console interaction using input() and print()
Conditional Logic	Menu choices and validation
Looping	Continuous menu operation
List Comprehension	Patient search matching
String Methods	strip() and lower() for input normalization

No third-party libraries are required by the supplied program. The code can be run using a standard Python 3 installation.

7. System Architecture

The supplied implementation follows a simple three-layer conceptual structure, although all layers are implemented inside one Python program.

Layer	Responsibility	Implementation
Presentation Layer	Displays menu, prompts and messages	input() / print() in main and methods
Application / Logic Layer	Performs patient operations	HospitalRecordSystem methods
Data / Model Layer	Represents individual patient records	Patient class + patients dictionary

Conceptual Architecture Diagram

User → Console Menu → HospitalRecordSystem → patients dictionary → Patient objects

The main() function acts as the controller. It creates the HospitalRecordSystem instance, presents the menu, receives the user's choice, and calls the corresponding method.

8. Data Model and Class Design

Patient Class

The Patient class is the data model. Each object stores the information associated with one patient.

Attribute	Meaning	Example
patient_id	Unique identifier	P101
name	Patient's name	Rahul Sharma
age	Patient age	35
gender	Patient gender	Male
doctor	Assigned doctor	Dr. Mehta
department	Assigned department	Cardiology
consultation_status	Current consultation state	Scheduled

HospitalRecordSystem Class

This class manages the collection of Patient objects and contains all major record operations.

Internal data structure: self.patients = {}

The dictionary uses patient_id as its key and the corresponding Patient object as its value.

9. Detailed Class Documentation

9.1 Patient.__init__()

Constructor that initializes a new patient object. The consultation_status parameter has a default value of **Scheduled**, so the caller does not have to explicitly provide a status.

Signature: __init__(patient_id, name, age, gender, doctor, department, consultation_status="Scheduled")

9.2 Patient.display_details()

Prints the patient's complete record using a formatted console layout. It does not return a value and is intended for human-readable display.

9.3 HospitalRecordSystem.__init__()

Initializes an empty dictionary named `patients`. Every patient added during the current program execution is stored in this dictionary.

9.4 `add_patient()`

Reads patient information from the console, checks whether the patient ID already exists, creates a `Patient` object, and inserts it into the dictionary.

Important behavior: if a duplicate ID is entered, the method stops before creating a new record.

9.5 `search_patient()`

Accepts a search query and compares it against `patient_id` or `name`. Both the stored values and query are converted to lowercase for case-insensitive matching.

The current implementation performs an exact match, not a partial or substring search.

9.6 `display_all_patients()`

Checks whether the dictionary is empty. If records exist, it loops through all `Patient` objects and calls `display_details()` for each one.

9.7 `assign_doctor_department()`

Locates a patient by ID and allows the user to replace the doctor and department. Pressing Enter without entering a new value keeps the current value.

9.8 `update_consultation_status()`

Displays four predefined status options. The entered option is translated through a `status_mapping` dictionary. If the user enters something other than 1–4, the entered text itself becomes the new status.

9.9 `delete_patient()`

Checks whether the supplied patient ID exists. If it does, the corresponding dictionary entry is deleted using `del`.

10. Functional Module Documentation

Module	Input	Processing	Output
Add Patient	ID, name, age, gender, doctor, department	Validate ID, status Validate ID, status Create Patient → store	Success / duplicate-ID message
Search	ID or name	Normalize and compare records	Matching patient details
Display All	None	Iterate through dictionary	All patient details
Assign Doctor	Patient ID + new doctor/department	Find object → update attributes	Update confirmation
Consultation Status	Patient ID + status choice	Map choice or accept custom status	Updated status
Delete	Patient ID	Check key → delete entry	Deletion confirmation

11. Program Workflow

Startup Workflow

- Python executes the file.
- The condition `if __name__ == '__main__':` becomes true when the file is run directly.
- `main()` is called.
- A `HospitalRecordSystem` object is created.

- An empty patient dictionary is initialized.
- The menu is repeatedly displayed.

Add Patient Workflow

Enter Patient ID → Check duplicate → Enter patient information → Apply default status if blank → Create Patient object → Store object in dictionary → Show confirmation.

Search Workflow

Enter ID/name → Convert query to lowercase → Iterate through patient values → Compare exact ID/name → Display all matching objects → If no match, show message.

Update Workflow

Enter Patient ID → Retrieve object from dictionary → If found, modify selected attributes → Print confirmation.

Delete Workflow

Enter Patient ID → Check dictionary membership → Delete record → Print confirmation.

12. Menu and User Interaction

Option	Action	Method Called
1	Add Patient	<code>hospital.add_patient()</code>
2	Search Patient	<code>hospital.search_patient()</code>
3	Display All Records	<code>hospital.display_all_patients()</code>
4	Assign / Update Doctor and Department	<code>hospital.assign_doctor_department()</code>
5	Update Consultation Status	<code>hospital.update_consultation_status()</code>
6	Delete Patient Record	<code>hospital.delete_patient()</code>
7	Exit	<code>break</code>

Any other input produces an invalid-selection message and returns to the menu.

13. Input Validation and Error Handling

The supplied program contains basic validation, but it is intentionally simple.

Validation	Current Behavior
Duplicate patient ID	Prevented before insertion
Missing consultation status	Defaults to Scheduled
Unknown patient ID during update	Shows Patient not found
Unknown patient ID during delete	Shows Patient ID not found
Invalid menu choice	Shows invalid selection message
Invalid status choice	Accepted as custom status
Age validation	Not implemented; stored as text
Empty name/doctor/department	Not explicitly rejected

For a production-oriented version, numeric validation, mandatory-field validation, controlled department/doctor values, input length checks, and structured exception handling should be added.

14. Sample Execution

The following is an illustrative console session showing the intended interaction.

```
==== Mini Hospital / Patient Record System ====
1. Add Patient
2. Search Patient
3. Display Patient Details (All Records)
4. Assign / Update Doctor and Department
5. Update Consultation Status
6. Delete Patient Record
7. Exit

Select an option (1-7): 1
Enter Patient ID: P101
Enter Patient Name: Rahul Sharma
Enter Age: 35
Enter Gender: Male
Enter Assigned Doctor: Dr. Mehta
Enter Assigned Department: Cardiology
Enter Consultation Status ... [Default: Scheduled]:
Patient 'Rahul Sharma' added successfully.

Select an option (1-7): 2
Enter Patient ID or Name to search: p101

=====
Patient ID       : P101
Name            : Rahul Sharma
Age             : 35
Gender          : Male
Assigned Doctor  : Dr. Mehta
Department      : Cardiology
Consultation Status : Scheduled
=====

Select an option (1-7): 5
Enter Patient ID to update consultation status: P101

Select Consultation Status:
1. Scheduled
2. In Progress
3. Completed
4. Cancelled
Enter option number or type a custom status: 2
Consultation status updated to: In Progress

Select an option (1-7): 7
Exiting system.
```

15. Algorithm / Pseudocode

Main Algorithm

START

Create HospitalRecordSystem

REPEAT

 Display menu

 Read choice

 IF choice = 1: add patient

 ELSE IF choice = 2: search patient

 ELSE IF choice = 3: display all

 ELSE IF choice = 4: update doctor/department

 ELSE IF choice = 5: update consultation status

 ELSE IF choice = 6: delete patient

 ELSE IF choice = 7: exit loop

 ELSE: display invalid choice

UNTIL exit

STOP

Add Patient Pseudocode

```
Read patient ID
IF patient ID already exists
    Display duplicate error
    RETURN
Read remaining patient details
IF status is empty
    status = Scheduled
Create Patient object
patients[patient_id] = Patient object
Display success message
```

Search Pseudocode

```
Read query
Convert query to lowercase
For each patient in patients:
    IF patient ID equals query OR patient name equals query:
        Display patient
If no matches:
    Display no-record message
```

16. Complexity Analysis

Operation	Approx. Time Complexity	Reason
Add Patient	$O(1)$ average	Dictionary membership check and insertion
Search Patient	$O(n)$	May inspect every stored patient
Display All	$O(n)$	Displays every record
Update Doctor/Department	$O(1)$ average	Dictionary lookup by patient ID
Update Status	$O(1)$ average	Dictionary lookup by patient ID
Delete Patient	$O(1)$ average	Dictionary deletion by key

Here, **n** represents the number of patient records currently stored. Dictionary operations are average-case $O(1)$, subject to normal Python dictionary behavior.

17. Testing and Test Cases

TC-01	Add unique patient	Valid ID and details	Patient added	Pass condition: record added
TC-02	Duplicate ID	Existing ID	Duplicate error	No new record created
TC-03	Search by ID	Existing ID	Patient details	Correct record displayed
TC-04	Search by name	Existing exact name	Patient details	Correct record displayed
TC-05	Search missing	Unknown query	No matching record	No crash
TC-06	Update doctor	Existing ID + doctor	Update confirmation	Doctor changes
TC-07	Update department	Existing ID + department	Update confirmation	Department changes
TC-08	Status update	Existing ID + option 2	In Progress	Status changes

TC-01	Add unique patient	Valid ID and details	Patient added	Pass condition: record ap
TC-09	Delete existing	Existing ID	Deletion confirmation	Record removed
TC-10	Delete missing	Unknown ID	Not found	No crash
TC-11	Empty status	Blank status	Scheduled	Default applied
TC-12	Invalid menu	e.g. 9	Invalid selection	Menu continues

18. Security and Data Considerations

Because this application represents a hospital record workflow, real deployments would require strong privacy and security controls. The supplied program does not implement those controls.

- No authentication or role-based access control is implemented.
- Patient data exists only in memory and disappears when the program terminates.
- There is no encryption at rest or in transit.
- There is no audit trail showing who changed a record.
- There is no database backup or recovery mechanism.
- There is no validation against malicious or malformed input beyond basic checks.
- The program should not be used with real patient/medical data without appropriate security, privacy, legal, and organizational controls.

For educational use, the application is useful for demonstrating record-management logic. For real healthcare use, the architecture would need to be redesigned around secure storage, authenticated access, authorization, auditing, encryption, backup, and applicable healthcare/privacy requirements.

19. Limitations

- Records are not persistent because they are stored only in RAM.
- There is no database integration.
- Search supports exact ID/name matching rather than partial search.
- Age is stored as a string and is not numerically validated.
- There is no standardized doctor or department master list.
- There is no patient history or visit-history model.
- There is no appointment date/time field.
- There is no billing, prescription, laboratory, or discharge workflow.
- There is no login system or user roles.
- The interface is console-based rather than graphical or web-based.
- Multiple users/computers cannot safely share the same in-memory records.

20. Future Enhancements

Enhancement	Proposed Improvement
Database	Use SQLite, PostgreSQL, or MySQL for persistent storage.
GUI	Build a desktop interface using Tkinter, PyQt, or another GUI framework.

Enhancement	Proposed Improvement
Web Application	Expose functionality through Flask/FastAPI/Django and a web frontend.
Authentication	Add secure login and role-based permissions.
Search	Support partial names, filters, department, doctor and status.
Appointments	Add appointment date, time, queue number and reminders.
Medical History	Separate patient identity from visit/clinical records.
Audit Logs	Record create/update/delete actions with timestamps and user identity.
Validation	Validate age, mandatory fields, allowed statuses and identifiers.
Reporting	Generate patient lists, consultation statistics and department reports.
Backup	Add automated backup and recovery procedures.
API Integration	Support controlled integration with other hospital systems.

21. Conclusion

The Mini Hospital / Patient Record System is a compact demonstration of how Python object-oriented programming can solve a practical record-management problem. The Patient class models individual records, while HospitalRecordSystem provides the operations required to manage those records.

The project demonstrates constructors, instance attributes, methods, dictionaries, list comprehensions, conditionals, loops, input handling, and basic validation. It also provides a clear starting point for a larger hospital management application.

The most important architectural limitation is the in-memory storage model. Extending the project with a database, authentication, validation, audit logging, and a graphical or web interface would transform the prototype into a more complete software system.

22. Complete Source Code Appendix

The following appendix reproduces the supplied program so that the documentation can be kept together with its implementation.

```
class Patient:
    def __init__(self, patient_id, name, age, gender, doctor, department, consultation_status="Scheduled"):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.gender = gender
        self.doctor = doctor
        self.department = department
        self.consultation_status = consultation_status

    def display_details(self):
        print("\n" + "=" * 40)
        print(f"Patient ID      : {self.patient_id}")
        print(f"Name           : {self.name}")
        print(f"Age            : {self.age}")
        print(f"Gender         : {self.gender}")
        print(f"Assigned Doctor : {self.doctor}")
        print(f"Department     : {self.department}")
        print(f"Consultation Status : {self.consultation_status}")
        print("=" * 40)

class HospitalRecordSystem:
    def __init__(self):
        self.patients = {}

    def add_patient(self):
        pid = input("Enter Patient ID: ").strip()
        if pid in self.patients:
            print("Error: A patient with this ID already exists.")
```

```

        return

    name = input("Enter Patient Name: ").strip()
    age = input("Enter Age: ").strip()
    gender = input("Enter Gender: ").strip()
    doctor = input("Enter Assigned Doctor: ").strip()
    department = input("Enter Assigned Department: ").strip()
    status = input(
        "Enter Consultation Status (e.g., Scheduled, In Progress, Completed) *  

        *[Default: Scheduled]: "
    ).strip()

    if not status:
        status = "Scheduled"

    self.patients[pid] = Patient(
        pid, name, age, gender, doctor, department, status
    )
    print(f"\nPatient '{name}' added successfully.")

def search_patient(self):
    query = input("Enter Patient ID or Name to search: ").strip().lower()
    matches = []
    for p in self.patients.values():
        if p.patient_id.lower() == query or p.name.lower() == query
    ]

    if not matches:
        print("No matching patient records found.")
        return

    for patient in matches:
        patient.display_details()

def display_all_patients(self):
    if not self.patients:
        print("No patient records available.")
        return

    for patient in self.patients.values():
        patient.display_details()

def assign_doctor_department(self):
    pid = input("Enter Patient ID to update doctor/department: ").strip()
    patient = self.patients.get(pid)

    if not patient:
        print("Patient not found.")
        return

    new_doctor = input(
        f"Enter New Doctor (Current: {patient.doctor}): "
    ).strip()
    new_department = input(
        f"Enter New Department (Current: {patient.department}): "
    ).strip()

    if new_doctor:
        patient.doctor = new_doctor
    if new_department:
        patient.department = new_department

    print("Doctor and Department assignment updated successfully.")

def update_consultation_status(self):
    pid = input("Enter Patient ID to update consultation status: ").strip()
    patient = self.patients.get(pid)

    if not patient:
        print("Patient not found.")
        return

    print("\nSelect Consultation Status:")
    print("1. Scheduled")
    print("2. In Progress")
    print("3. Completed")
    print("4. Cancelled")
    choice = input(
        "Enter option number or type a custom status: "
    ).strip()

    status_mapping = {
        "1": "Scheduled",
        "2": "In Progress",
        "3": "Completed",
        "4": "Cancelled"
    }

```

```

        patient.consultation_status = status_mapping.get(choice, choice)
        print(
            f"Consultation status updated to: "
            f"{patient.consultation_status}"
        )

    def delete_patient(self):
        pid = input("Enter Patient ID to delete: ").strip()
        if pid in self.patients:
            del self.patients[pid]
            print(f"Patient record {pid} has been removed.")
        else:
            print("Patient ID not found.")

def main():
    hospital = HospitalRecordSystem()

    menu = """
==== Mini Hospital / Patient Record System ====
1. Add Patient
2. Search Patient
3. Display Patient Details (All Records)
4. Assign / Update Doctor and Department
5. Update Consultation Status
6. Delete Patient Record
7. Exit
"""

    while True:
        print(menu)
        choice = input("Select an option (1-7): ").strip()

        if choice == "1":
            hospital.add_patient()
        elif choice == "2":
            hospital.search_patient()
        elif choice == "3":
            hospital.display_all_patients()
        elif choice == "4":
            hospital.assign_doctor_department()
        elif choice == "5":
            hospital.update_consultation_status()
        elif choice == "6":
            hospital.delete_patient()
        elif choice == "7":
            print("Exiting system.")
            break
        else:
            print(
                "Invalid selection. Please choose an option between 1 and 7."
            )

if __name__ == "__main__":
    main()

```

End of Documentation